



MyScoutee

OFFICIAL REGISTRY DOCUMENTATION

OPERATIONS DOCUMENTATION

MyScoutee Registry Operations Manual

Installation, commissioning, trust verification, operation, recovery, and retirement of the standalone MyScoutee Registry 1.0.0 release.

TECHNOLOGY SENSITIVITY

The combined deployment topology, trust and key-handling procedures, release and recovery controls, and operational implementation details reduce the cost of reconnaissance for an unauthorized attacker. Distribute only through approved internal channels. Confidentiality supplements—never replaces—access control, signed artifacts, encryption, key protection, monitoring, and tested recovery.

Document ID MSC-ROM-001	Document version 1.0.0
Applicable software MyScoutee Registry 1.0.0	Status RELEASED BASELINE
Release date 2026-07-30	Platform Debian-compatible Linux, amd64

Document Control

Audience	Registry host administrators, Registry operators, security reviewers, release custodians, backup/recovery personnel, and incident responders.
Normative operator release	myscoutee-registry_1.0.0_amd64.deb, containing the installed systemd and Docker Compose model plus exactly two production images.
Production images	myscoutee-registry:1.0.0-prod and myscoutee-registry-nginx:1.0.0-prod.
Delivery channel	GitHub Releases assets with independently authenticated release-signing material; no MyScoutee APT repository is part of version 1.0.0.
Classification	Confidential — Internal Distribution Only. Registry binaries, release artifacts, configuration, key-handling procedures, and operational details must remain inside approved internal distribution and access controls.
Review triggers	Any change to release artifacts, image content, Compose topology, systemd behavior, public protocol, scope/key rules, state format, TLS handling, backup procedure, or trust-verification tooling.

Version history

Version	Date	Status	Change
1.0.0	2026-07-30	Released baseline	Initial standalone VM release with Debian package, two bundled production images, client tools, host qualification, and separate external deployment verification.

Controlled evidence base

This manual is tied to the following version-controlled surfaces:

- packaging/compose/, packaging/debian/, packaging/systemd/, packaging/templates/, and packaging/scripts/;
- packaging/client-tools/ and packaging/verify-deployment/;
- the Go registry implementation and its embedded state-integrity rules;
- the companion MyScoutee Registry Protocol Developer Manual v1.0.0.

The repository tools/ directory contains developer utilities. It is not an operator release surface and is not an installation dependency.

Version 1.0.0 is a base release

These procedures define a clean 1.0.0 installation. They do not define a predecessor installation layout, transition wrapper, or old-state conversion contract. Do not copy historical source deployment instructions into this release procedure.

Table of Contents

- | | |
|---|--|
| 1. Purpose and safety boundary | 10. Lifecycle, health, and monitoring |
| 2. Deployment modes and responsibilities | 11. Registry-local administrative CLI |
| 3. Architecture and trust model | 12. Backup and restore |
| 4. Host and network requirements | 13. Reinstall, future update, and rollback |
| 5. Release artifacts and authenticity | 14. Security operations and incidents |
| 6. Standalone VM installation | 15. Retirement and removal |
| 7. Configuration, scope, keys, and state | 16. Known limits |
| 8. DNS, TLS, firewall, and time | 17. Quick reference |
| 9. Host qualification and external verification | |

1. Purpose and Safety Boundary

This manual explains the supported operator path for one independently governed MyScoutee Registry 1.0.0 instance on a standalone Linux host or virtual machine. The installed service accepts signed MyScoutee deployment records, protects an append-only SQLite history, and exposes registry-signed evidence over HTTPS.

In scope

- installation from the released Debian package;
- the separate Registry and Nginx production containers managed by systemd and Docker Compose;
- immutable registry scope, Ed25519 registry identity, VOPRF keyring, TLS, and persistent SQLite state;
- host qualification, independent public verification, monitoring, local administrative commands, backup, restore, incidents, and removal;
- controlled artifact delivery and explicit trust pinning.

Out of scope

- multi-node operation, automatic failover, cross-registry synchronization, or active-active SQLite;
- unattended Registry self-update or a public MyScoutee APT repository;
- building production artifacts on the operator host;
- automatic repair of inconsistent signed or append-only state;
- legal, tax, payment, acquisition, or beneficial-owner decisions.

Technical evidence is not payment authority

Registry settlements, reviews, ownership transfers, and exit allocations are signed technical records. They do not execute payments, create invoices, decide tax treatment, or replace external legal approval.

2. Deployment Modes and Responsibilities

Mode	Purpose	Operational boundary
Standalone VM/runtime-engine release	The supported operator installation described by this manual.	Install myscoutee-registry_1.0.0_amd64.deb. The package supplies the two production images, Compose model, systemd unit, host qualifier, and client tools. No source checkout or build toolchain is needed on the target.
Direct production instance	A production instance operated directly inside an organization-owned runtime environment.	It uses the same released production image roles and protocol trust rules, but its infrastructure lifecycle can be owned by the central platform team. Do not assume the Debian paths or commands apply unless that instance was installed from the Debian package.
Development instance	Source development, debugging, fixtures, and rapid iteration.	May use developer Compose files, development images, seeded data, and repository tools/. It is not release evidence and must never share scope, keys, state, DNS, backups, or client bindings with production.

Measure the production path

Release qualification and user-facing performance measurements must use production images without source synchronization, hot deployment, or frontend/backend development watchers.

3. Architecture and Trust Model

Nginx service

Terminates TLS, redirects HTTP to HTTPS, enforces body, connection, and request-rate limits, and forwards to the Registry over the internal Docker network.

Registry service

Runs the non-root scratch Go image, verifies signed requests and state integrity, signs receipts and public evidence, and writes only to the protected data bind mount.

Persistent state

`/var/lib/myscoutee-registry/data` contains SQLite, the generated Registry signing key, and the VOPRF keyring. These files form one recovery unit.

Protected configuration

`/etc/myscoutee-registry/registry.env` and `/etc/myscoutee-registry/tls/` hold host-specific configuration and TLS material. They are never release payload secrets.

Three independent trust checks

1. **Release trust:** authenticate the Debian artifact through its Ed25519 release metadata and a full SPKI fingerprint obtained out of band.
2. **Transport trust:** validate the HTTPS certificate chain, hostname, validity period, and final origin without insecure TLS or redirect following.
3. **Registry identity trust:** verify the self-signed identity response, derive its `rkey_...` key ID, and pin the approved origin, scope, and registry key independently of TLS.

A valid TLS certificate does not establish the expected Registry signing identity. A valid Registry self-signature proves possession of the advertised key but does not prove that the endpoint is the intended deployment. Both checks and the out-of-band pin are required.

Scope is permanent identity domain separation

`REGISTRY_SCOPE` is recorded with the Registry identity on first start and is included in signed protocol state. A different scope represents a different sovereign namespace. Changing the environment value against an established database must fail startup; it is not a rename procedure.

4. Host and Network Requirements

Area	Requirement	Initial check
Host	Debian-compatible amd64 Linux host or VM with systemd and administrative access.	<code>dpkg --print-architecture</code> must report amd64.
Container runtime	Docker Engine and Docker Compose v2. Install the local package through APT so declared dependencies can be resolved.	<code>docker info</code> and <code>docker compose version</code> .
Storage	Durable local filesystem with reliable locks, atomic rename, directory sync, and SQLite WAL/fsync behavior. Unqualified network filesystems are prohibited.	Power-loss and clean-host restore drill.
Time	Reliable UTC synchronization. Signed requests default to a five-minute accepted skew and signed state rejects clock rollback.	<code>timedatectl status</code> or platform equivalent.
Network	Stable DNS, certificate-valid HTTPS, intended inbound client reachability, and a management path separated from public traffic.	External DNS, TLS, firewall, and redirect checks.
Backup	Encrypted backup storage and an isolated restore target able to preserve numeric ownership and modes.	Recorded restore evidence.

The repository defines no qualified hardware minimum or throughput SLA. A sensible initial planning point is 2 vCPU, 2 GiB RAM, and 20 GiB SSD for small controlled use, or 4 vCPU, 4–8 GiB RAM, and 50–100 GiB SSD for initial production planning. Measure actual ledger, nonce, identity-presence, and health costs, and retain at least the locally approved free-space margin.

5. Release Artifacts and Authenticity

The version 1.0.0 operator release is delivered as GitHub Releases assets, not through a MyScoutee APT repository. The operational payloads are:

- `myscoutee-registry_1.0.0_amd64.deb`;
- `myscoutee-registry-client-tools_1.0.0.tar.gz`;
- publisher-supplied release metadata and an Ed25519 public key.

The package builder creates the Debian package and client-tools archive. The Debian package carries a byte-identical copy of that versioned archive at `/usr/share/myscoutee-registry/myscoutee-registry-client-tools_1.0.0.tar.gz`; it does not install expanded client scripts on the Registry host. Publisher signing is a separate release-authority step. The client tool `verify-release.sh` accepts one artifact, the release manifest, the release public key, and the independently trusted full SPKI fingerprint. It verifies:

- canonical Ed25519 SPKI encoding and the complete sha256: fingerprint;
- the deterministic `pkey_...` signing key ID;
- artifact filename, purpose, version, byte size, and SHA-256 digest;
- the Ed25519 signature over the canonical artifact metadata domain.

```
./verify-release.sh \  
myscoutee-registry_1.0.0_amd64.deb \  
/secure/release-manifest.json \  
/secure/release-public-key.pem \  
sha256:<64-lowercase-hex-characters>
```

A co-located key is not a trust anchor

Do not trust a release key merely because it was downloaded beside the package. Obtain and compare the complete SPKI fingerprint through an independent authenticated channel. Refuse any artifact for which signed release verification cannot be completed.

The Debian package embeds both production image archives. During package configuration, their archive SHA-256 values and portable content fingerprints are verified before the images are accepted. The package contains no production environment, TLS private key, Registry signing key, VOPRF seed, or SQLite database.

6. Standalone VM Installation

6.1 Direct host installation

1. Verify the downloaded package as described in §5.
2. Install it through local APT so Docker and Compose dependencies can be resolved.
3. Review and replace every deployment-specific placeholder before the first start.
4. Provision the TLS certificate and private key at the configured paths.
5. Start explicitly, run host qualification, then run the external verifier from another trust boundary.

```
sudo apt install ./myscoutee-registry_1.0.0_amd64.deb

sudoedit /etc/myscoutee-registry/registry.env
sudo install -o root -g root -m 0600 /secure/fullchain.pem \
/etc/myscoutee-registry/tls/fullchain.pem
sudo install -o root -g root -m 0600 /secure/privkey.pem \
/etc/myscoutee-registry/tls/privkey.pem

sudo systemctl start myscoutee-registry
sudo /opt/myscoutee-registry/packaging/scripts/qualify-deployment.sh \
check --expected-version 1.0.0 --timeout-seconds 60
```

Package installation deliberately does not start the Registry

The package loads and verifies both images, creates protected directories, and enables the systemd unit. It does not start or restart the Registry and does not run the external verifier. This prevents the placeholder scope, hostname, or TLS paths from becoming the first-start identity.

6.2 Remote client-tools flow

Obtain the separately released `myscoutee-registry-client-tools_1.0.0.tar.gz`, verify it, and extract it on the administrative client. A host that already has the Debian package installed also exposes the same archive bytes at `/usr/share/myscoutee-registry/myscoutee-registry-client-tools_1.0.0.tar.gz` for controlled retrieval. Do not run client wrappers from the Registry host. Copy `install.env.example` from the extracted archive to the gitignored `install.env`, restrict it to mode 0600, and configure SSH transport and release verification.

```
cp install.env.example install.env
chmod 0600 install.env
./install.sh /path/to/myscoutee-registry_1.0.0_amd64.deb
```

START_SERVICE=false is the default. For a fresh host, an explicit START_SERVICE=true requires all three validated local inputs together: DEPLOYMENT_ENV_FILE, TLS_CERTIFICATE_FILE, and TLS_PRIVATE_KEY_FILE. The wrapper transfers them through a protected temporary directory, installs root-owned configuration/TLS files, and then runs the package-provided restart-and-check qualification action. The actual install.env, deployment environment, and TLS files are gitignored and excluded from release artifacts.

The client-tools archive contains:

- install.sh, install.env.example, and deployment.env.example;
- remote-transport.sh and forward-localhost.sh;
- verify-release.sh and the separate verify-deployment/ verifier;
- purge.sh for an explicit destructive package purge.

External verification is never folded into installation

The remote installer can print the command for the requested public URL, but it does not execute it. Run that command separately from the client after the host installation and host qualification have completed.

6.3 Retrieve client tools and open an SSH tunnel

An authorized Registry operator with SFTP access can download the exact version-matched client-tools archive from an already installed host to an administrative workstation. Use the workstation's approved SSH configuration; do not place a password or private-key content in the command:

```
sftp registry-host
sftp> get /usr/share/myscoutee-registry/myscoutee-registry-client-tools_1.0.0.tar.gz
./myscoutee-registry-client-tools_1.0.0.tar.gz
sftp> bye
```

SFTP provides transport, not release authenticity. Authenticate the downloaded archive against the release metadata and independently pinned release-key SPKI fingerprint described in §5 before extraction. Then extract and run the helpers only on the workstation:

```
mkdir -m 0700 registry-client-tools-1.0.0
tar -xzf myscoutee-registry-client-tools_1.0.0.tar.gz \
  -C registry-client-tools-1.0.0
cd registry-client-tools-1.0.0

cp install.env.example install.env
chmod 0600 install.env
./forward-localhost.sh
```

Use the extracted `verify-deployment/run.mjs` from the workstation for the separate external check. Never extract or execute `install.sh`, `forward-localhost.sh`, or the verifier on the Registry server. SSH tunnel authentication does not replace public TLS validation or the expected Registry scope/key pin.

7. Configuration, Scope, Keys, and State

The protected file is `/etc/myscoutee-registry/registry.env`. Keep it owned by root, mode 0600, and never replace it with a symlink. At minimum, confirm these values before first start:

```
MYSCOUTEE_REGISTRY_VERSION=1.0.0
REGISTRY_PRODUCTION_IMAGE=myscoutee-registry:1.0.0-prod
REGISTRY_NGINX_PRODUCTION_IMAGE=myscoutee-registry-nginx:1.0.0-prod

REGISTRY_SCOPE=production:primary
REGISTRY_DEMO_SEED=false

REGISTRY_PUBLIC_HOSTNAME=registry.example.com
REGISTRY_PUBLIC_BIND_ADDRESS=0.0.0.0
REGISTRY_HTTP_PORT=80
REGISTRY_HTTPS_PORT=443
REGISTRY_TLS_CERTIFICATE_PATH=/etc/myscoutee-registry/tls/fullchain.pem
REGISTRY_TLS_PRIVATE_KEY_PATH=/etc/myscoutee-registry/tls/privkey.pem
```

Setting	Release default	Operator rule
REGISTRY_SCOPE	Placeholder in the installed example	Replace before first start. Choose once; a mismatch with established state must fail.
REGISTRY_DATABASE_PATH	/data/registry.db	Keep inside the protected data bind mount.
REGISTRY_SIGNING_KEY_PATH	/data/registry-signing-key.pem	Generated Ed25519 identity; restore only with its matching database and scope.
REGISTRY_GENERATE_SIGNING_KEY	true	Allow only for the deliberate pristine first start; set false after commissioning.
REGISTRY_GLOBAL_IDENTITY_VOPRF_KEYRING_PATH	/data/registry-global-identity-voprf-keyring.json	Sensitive seed keyring; back up with SQLite and the Registry signing key.
REGISTRY_GENERATE_GLOBAL_IDENTITY_VOPRF_KEY	true	Allow only for first initialization; set false after commissioning.
REGISTRY_TIMESTAMP_SKEW	5m	Fix clocks instead of widening casually.
REGISTRY_CHECKPOINT_INTERVAL	1m	Completed UTC-day checkpoint attempt interval.
REGISTRY_MAX_REQUEST_BODY_BYTES	65536	Coordinate with REGISTRY_NGINX_MAX_BODY_SIZE.
REGISTRY_NGINX_REQUEST_RATE	10r/s	Per-observed-address rate. Review shared-NAT traffic before changing.
REGISTRY_NGINX_REQUEST_BURST	20	Burst accepted before HTTP 429.
REGISTRY_NGINX_CONNECTION_LIMIT	20	Concurrent connections per observed address.

First-start lock-down

After the first successful start and after recording the Registry identity, set both generation switches to false. Restart and qualify so loss of the data directory fails closed instead of silently creating a new identity:

```
sudoedit /etc/myscoutee-registry/registry.env
sudo systemctl restart myscoutee-registry
sudo /opt/myscoutee-registry/packaging/scripts/qualify-deployment.sh \
  check --expected-version 1.0.0 --timeout-seconds 60
```

The release uses a bind mount at `/var/lib/myscoutee-registry/data`, not a disposable container layer. The directory is owned for container UID/GID 65532 and must not be manually edited. SQLite uses WAL, foreign keys, full synchronization, serialized access, append-only triggers, and integrity verification. A failed identity, chain, signature, direct-row, Merkle, or checkpoint check must fail closed.

8. DNS, TLS, Firewall, and Time

DNS and TLS

- Use one stable hostname covered by the certificate. A Registry base URL is an origin, not a path, query, fragment, or credential-bearing URL.
- Nginx accepts TLS 1.2/1.3, disables session tickets, adds HSTS and X-Content-Type-Options: nosniff, and redirects HTTP to the configured HTTPS hostname and port.
- The Registry container remains plain HTTP on the internal Docker network and publishes no host port. Nginx is the only host-facing service.
- Replace the certificate and key as one protected pair, restart Nginx through systemd, and verify the public chain and hostname normally. Do not use `-k`.

```
sudo systemctl restart myscoutee-registry

openssl s_client \
  -connect registry.example.com:443 \
  -servername registry.example.com \
  < /dev/null

curl --fail --show-error https://registry.example.com/versionz
```

Firewall baseline

Flow	Default	Rule
MyScoutee deployments → HTTPS	TCP 443	Allow the intended client networks. Account for shared NAT and rate limits.
HTTP redirect	TCP 80	Optional exposure; it performs a 308 redirect only. Clients should be configured with HTTPS directly.
Registry backend	TCP 8080 on the internal Docker network only	Do not publish a host port or create a firewall bypass around Nginx.
SSH/management	Site policy	Restrict separately from protocol traffic and audit privileged access.

Time

Monitor UTC synchronization continuously. If the host clock is wrong, stop signed writes, correct the approved time source, restart and verify. Never rewrite accepted timestamps in SQLite.

9. Host Qualification and External Verification

9.1 Package-provided host qualification

The host qualifier validates the installed package version, two-image manifest, portable image content fingerprints, configured image references, systemd state, container image references and health, and the local TLS/Nginx path's version, health, and identity consistency. It uses the configured certificate and hostname without exposing the Registry backend on a host port.

```
sudo /opt/myscoutee-registry/packaging/scripts/qualify-deployment.sh \
  check --expected-version 1.0.0 --timeout-seconds 60
```

Use `restart-and-check` only when an explicit restart is part of the approved action. Host qualification validates the configured certificate and local Nginx route, but it does not prove public DNS, external routing, or the expected business trust pin.

9.2 Separate external deployment verifier

Run the URL-driven verifier from the client side after the deployment is already running:

```
node verify-deployment/run.mjs \
  --url https://registry.example.com \
  --expected-version 1.0.0 \
  --expected-scope production:primary \
  --expected-key-id rkey_0123456789abcdef0123456789abcdef
```

The verifier is a manual operator command, not an automated E2E test, post-install hook, or aggregate test-runner member. It never starts or changes the deployment. It sends exactly one unauthenticated GET to each of:

- `/versionz` — exact service, build version, and protocol version;
- `/v1/registry/identity` — canonical Ed25519 SPKI, deterministic key ID, self-signature, and immutable scope;
- `/healthz` — integrity status, scope/key consistency, ledger extent, and head invariants.

It requires an HTTPS origin, follows no redirects, refuses insecure TLS configuration, bounds response bodies, and performs no retries. For a private CA, extend normal Node trust with `NODE_EXTRA_CA_CERTS`. The `--allow-http` option is restricted to localhost or a literal loopback address for an intentional tunnel/VM test and cannot weaken a production URL.

Health is not a cheap ping

/healthz performs integrity work. Do not wrap the external verifier in a high-frequency monitor or retry loop. Use it for commissioning, release acceptance, and deliberate integrity checks.

9.3 Commissioning record

- ☐ Debian package version, SHA-256, release-signing key ID, and full pinned release-key SPKI fingerprint.
- ☐ Both production image references and installed content fingerprints.
- ☐ Host/kernel/Docker/Compose versions, filesystem, time status, firewall rules, and public origin.
- ☐ Certificate chain and expiry.
- ☐ Exact registry_scope, registry_key_id, full Registry SPKI fingerprint, and identity response.
- ☐ Initial ledger index, entry count, and head hash.
- ☐ Host qualification result and separate external verifier result.
- ☐ Backup destination and clean-host restore schedule.

10. Lifecycle, Health, and Monitoring

Common lifecycle commands

```
sudo systemctl start myscoutee-registry
sudo systemctl stop myscoutee-registry
sudo systemctl restart myscoutee-registry
sudo systemctl status myscoutee-registry

sudo /opt/myscoutee-registry/packaging/scripts/registry-compose.sh ps
sudo /opt/myscoutee-registry/packaging/scripts/registry-compose.sh \
  logs --tail 200 registry nginx
```

The installed Compose helper fixes the project name, protected environment, and installed Compose file. Use it instead of reconstructing raw Docker Compose arguments. Do not add build or pull behavior to the installed release.

Monitoring minimum

Signal	Alert condition	Response
systemd/container state	Inactive, unhealthy, restart loop, or unexpected image reference.	Preserve logs; check state, image fingerprint, TLS, and integrity errors.
External HTTPS	Certificate, /versionz, identity, expected scope/key, or sustained latency failure.	Compare with commissioning evidence before restoring traffic.
Disk/inodes	Locally approved headroom breached.	Expand or restore through a qualified storage process; never delete append-only rows or durable nonces.
Backups	Missed backup, encryption/upload failure, or stale restore evidence.	Create and test a complete recovery unit.
Clock	NTP unsynchronized or offset approaches request skew.	Stop signed writes and repair the time source.
HTTP behavior	Rising 429, 5xx, signature, replay, or request-size failures.	Separate abuse, shared-NAT concentration, client defect, and integrity fault.
Ledger/checkpoints	Head regression, missing completed-day checkpoint, or proof failure.	Stop writes and follow the integrity incident procedure.

The Go service writes structured JSON logs to stdout. Configure Docker/host rotation and a protected collector. Do not capture private administrative command output in shared logs or support tickets.

11. Registry-local Administrative CLI

Run Registry commands inside the installed production container so they use the exact protected scope, signing key, VOPRF keyring, and SQLite state:

```
sudo /opt/myscoutee-registry/packaging/scripts/registry-compose.sh \  
  exec -T registry /registry COMMAND [OPTIONS]
```

Use `/registry COMMAND --help` as the flag authority. Copy opaque identifiers, event hashes, cursors, review targets, and idempotency keys exactly. Existing-state commands must fail if the database, key, scope, or integrity boundary is missing or inconsistent.

Rail	Representative commands	Boundary
Build information	<code>version</code>	Config-free JSON build and protocol version.
Announcements	<code>publish-announcement</code>	Strict append-only local publication; no public mutation endpoint.
Company claims	<code>list-operator-claims</code> , <code>show-operator-claim</code> , <code>approve/reject/suspend/reinstate</code> commands	Review, private detail, and independent eligibility rails. Stale targets fail closed.
Cases and exits	<code>case</code> , <code>exit-review</code> , <code>ownership-transfer</code> , and <code>final-allocation</code> command families	Signed technical governance evidence; no payment or legal execution.
Accounting	<code>leaderboard</code> , <code>revenue</code> , <code>calculate-settlement</code> , <code>settlements</code>	Some output contains protected business amounts.
Global identity	<code>global-identity-dedup</code> , <code>rotate-global-identity-key</code>	VOPRF material is sensitive; rotation requires backup and restart.
Merkle evidence	<code>proof</code> and <code>consistency</code> command families	Generate or verify signed append-only evidence.

Docker access is administrative authority

Restrict and audit host/Docker access. Registry signatures show what this authority recorded; they do not authenticate a separate reviewer. Claim detail, settlement, and exit output can contain sensitive business or contact data.

12. Backup and Restore

Complete recovery unit

Back up these items together under one evidence ID:

- every file under `/var/lib/myscoutee-registry/data`, including SQLite/WAL/SHM when present, Registry signing key, and VOPRF keyring;
- `/etc/myscoutee-registry/registry.env` and the TLS certificate/private-key pair under their custody policy;
- installed package version, package hash/signing evidence, image content fingerprints, and host/platform record;
- Registry identity, latest health head, selected checkpoints/Merkle evidence, and backup checksums.

Raw copy requires quiescence

The release has no Registry-native backup command, and the scratch runtime has no shell or SQLite utility. Do not copy a changing database, WAL, and SHM sequentially. Stop the service, use a separately qualified SQLite online-backup tool, or take an atomic storage snapshot.

Quiesced backup

1. Record UTC time, package/image identity, public identity, ledger head, and service state.
2. Stop public ingress and run `sudo systemctl stop myscoutee-registry`.
3. Confirm the exact bind directory `/var/lib/myscoutee-registry/data`; never guess another path.
4. Take an atomic snapshot or root-controlled archive preserving numeric UID/GID, modes, timestamps, and every file.
5. Back up configuration, TLS, and release evidence under the same backup ID.
6. Encrypt, checksum, store, and independently verify the backup.
7. Restart, run host qualification, and verify the public identity/head.

Clean-host restore

1. Isolate the target from client traffic and install the exact recorded Debian release without starting it.
2. Restore the complete data directory with numeric owner 65532 and original modes.
3. Restore protected configuration and TLS with the original scope. Keep generation switches false.

4. Start the service and preserve startup logs. Integrity, key, scope, and state mismatch must fail.
5. Run host qualification, then compare `/versionz`, identity, health, ledger head, and selected proof evidence with the backup record.
6. Run the external verifier and restore traffic gradually.

A replacement generated key or an empty healthy ledger is not recovery. Restore drills must be performed on an isolated clean host at the frequency required by the local RPO/RTO.

13. Reinstall, Future Update, and Rollback

The Registry has no self-updater. Its announcement feed concerns MyScoutee deployment packages; Registry package replacement is an explicit host-administration action.

Same-version reinstall

A 1.0.0 reinstall refreshes the package payload while preserving configuration and state. Package configuration still does not restart an active Registry. The client wrapper uses a controlled `dpkg --unpack` plus APT dependency repair for this case and restarts only when explicitly requested.

Future release acceptance

1. Authenticate the new Debian artifact and client tools through their release-signing evidence.
2. Review release notes, state-format changes, image contents, Nginx policy, protocol changes, and release qualification.
3. Take and verify a complete pre-change recovery-unit backup and rehearse it on an isolated host.
4. Schedule downtime or drain ingress; install without assuming package configuration restarts the service.
5. Explicitly restart, run host qualification, and compare scope, Registry key, ledger head, public TLS, representative reads, and proofs.
6. Run the separate external verifier and retain the prior backup until the observation window closes.

Rollback is a recovery decision

If a future binary has changed persistent state, do not simply start an older image against that state. Restore the complete pre-change database/key/keyring/configuration unit with the corresponding released package unless that exact downgrade path was explicitly qualified.

14. Security Operations and Incidents

Preventive controls

- least-privilege host/Docker access, MFA on infrastructure control planes, a restricted management network, and privileged-command audit;
- mode-0600 configuration and keys, no symlink substitution, encrypted backups, and no production secrets in artifacts or source;
- pinned release metadata, full release-key SPKI fingerprint, pinned Registry identity, controlled TLS renewal, and reliable UTC time;
- external retention of selected identity, checkpoint, and Merkle evidence;
- no manual SQLite UPDATE/DELETE, nonce pruning, or query-table repair;
- controlled distribution of Registry binaries and artifacts because exposure increases reverse-engineering, vulnerability-response, provenance, and patch-coordination risk even though security must not depend on binary secrecy.

Incident	Immediate action	Recovery boundary
Integrity/health failure	Stop public ingress and writes; preserve logs, state snapshot, image fingerprints, time evidence, and checkpoints.	Do not repair signed rows manually. Diagnose or restore a complete verified recovery unit.
SQLite/storage corruption	Stop both containers and take a forensic snapshot before any tool touches the data.	Restore database, Registry key, and VOPRF keyring together.
Registry key loss	Keep generation disabled and stop serving.	Restore the matching key. A new key is a new Registry identity.
Registry key compromise	Isolate host, preserve evidence, stop signing, and notify client operators through an independent channel.	Version 1 has no in-place Registry identity rotation; commission a new identity and govern client transition explicitly.
VOPRF keyring exposure	Stop global-identity operations and restrict affected records/backups.	Rotation limits future use but does not undo old seed/commitment exposure.
TLS/DNS compromise	Revoke/replace credentials, isolate public ingress, and compare Registry identity out of band.	Registry state may remain valid only after key/state verification.
Clock rollback/drift	Stop signed writes and repair the approved time source.	Restart and verify; never rewrite stored timestamps.
Disk full	Stop writes and expand or restore storage through a qualified process.	Never delete append-only history, durable nonces, Merkle nodes, or direct rows.
Release artifact/key concern	Stop new installation, preserve downloaded assets and metadata, revoke distribution access, and validate the pinned signing key out of band.	Reissue only under an authenticated release decision; do not silently replace an existing GitHub asset.

15. Retirement and Removal

1. Freeze new enrollment and notify bound client operators through a trusted channel.
2. Complete signed deactivation/disconnect where required while the original Registry identity is still available.
3. Finalize required checkpoints, export agreed evidence, and verify a complete final backup on an isolated host.
4. Stop the service and revoke public DNS/TLS/firewall exposure.
5. Apply the approved retention/destruction policy to state, keys, TLS, configuration, logs, images, packages, and backups.

Package removal versus purge

`apt remove myscoutee-registry` stops/disables the stack but preserves host configuration and state. `apt purge myscoutee-registry` is deliberately destructive: it removes Registry configuration, TLS material, SQLite state, Registry/VOPRF keys, installed payload, and the configured managed images. Independently stored backups are not removed.

Purge is irreversible without a verified backup

Do not use the client `purge.sh` helper as a troubleshooting shortcut. Obtain explicit authorization, verify the exact target and final backup, and record what will be destroyed before running it.

16. Known Limits

Limit	Operational treatment
Single-node SQLite; no replication/failover protocol	Define downtime, RPO, and RTO; maintain tested clean-host recovery.
No Registry-native backup command	Use qualified quiesced copy, SQLite online backup, or atomic storage snapshots.
No Registry self-updater	Authenticate and install every release explicitly under change control.
Publisher signing is outside the package builder	Require release metadata and the out-of-band full SPKI fingerprint before accepting published GitHub assets.
No repository-provided OCI signature, SBOM publication, or public immutable-image policy	Keep artifact distribution controlled and preserve signed package-level evidence; apply the release authority's additional supply-chain controls.
Health performs integrity work	Avoid high-frequency public polling and measure cost as history grows.
No in-place Registry identity rotation or HSM/threshold implementation	Protect the Registry key as a root trust asset; key compromise requires a separately commissioned identity.
VOPRF seed holder can test likely low-entropy inputs against restricted commitments	Separate, minimize, and audit access to VOPRF material and protected records.
No cross-registry federation	Do not present another Registry as transparent fallback or merged history.
No formal capacity/SLA matrix	Benchmark expected registration, report, identity, announcement, proof, and health traffic before launch.

17. Quick Reference

Release artifacts

myscoutee-registry_1.0.0_amd64.deb
myscoutee-registry-client-
tools_1.0.0.tar.gz
release manifest
Ed25519 release public key +
independently pinned full SPKI fingerprint

Production images

myscoutee-registry:1.0.0-prod
myscoutee-registry-nginx:1.0.0-prod

Host paths

Install: /opt/myscoutee-registry
Config/TLS: /etc/myscoutee-registry
State: /var/lib/myscoutee-
registry/data
Client archive only:
/usr/share/myscoutee-
registry/myscoutee-registry-client-
tools_1.0.0.tar.gz

Public checks

GET /versionz
GET /v1/registry/identity
GET /healthz
HTTP → HTTPS 308

Never expose

Any host-published Registry backend port
SQLite/WAL/SHM
Registry, VOPRF, TLS, or release private
keys
Protected environment
Private CLI output

Emergency stop

sudo systemctl stop myscoutee-
registry
Preserve logs, exact state directory,
package/image evidence, time evidence,
and current identity before diagnosis.

Commissioning checklist

- ☐ Release artifact verified against signed metadata and an independently pinned full SPKI fingerprint.
- ☐ Package installed through local APT; service remained stopped until provisioning was complete.
- ☐ Non-placeholder scope, public hostname, bind address, ports, and TLS paths reviewed.
- ☐ Certificate-valid HTTPS and firewall behavior verified externally.

- ☐ Host qualifier passed for package 1.0.0 and both managed image content fingerprints.
- ☐ /versionz, scope, Registry key ID, full Registry SPKI fingerprint, and ledger head recorded.
- ☐ Generation switches disabled after first initialization and the service requalified.
- ☐ External verifier passed separately with expected version, scope, and key ID pins.
- ☐ Encrypted complete backup and isolated restore drill scheduled.

Companion protocol manual

For wire formats, canonical messages, signatures, HTTP rules, idempotency, response contracts, and client integration, use **MyScoutee Registry Protocol Developer Manual v1.0.0** (MyScoutee_Registry_Protocol_Developer_Manual_v1.0.0_EN.pdf).

MyScoutee Registry Operations Manual · MSC-ROM-001 · Version 1.0.0 · 2026-07-30 · Confidential — Internal Distribution Only